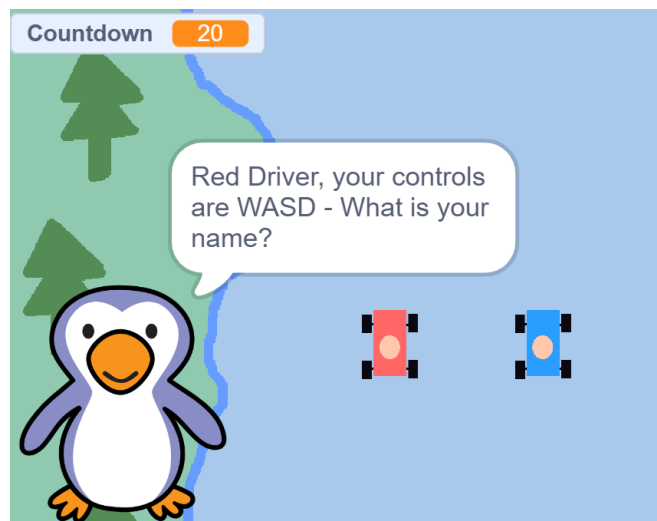


# GLACIER RACE

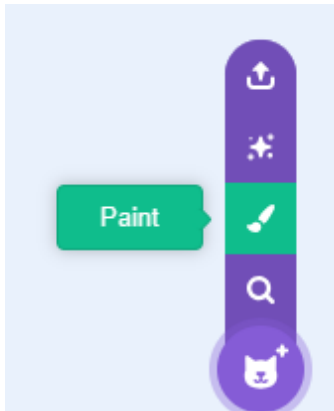


# GLACIER RACE

Glacier race is a two-player game where a red car vs a blue car race up the screen, swerving around obstacles and collecting gems as you go. There's no finish line in this race – the winner is simply the person with the most gems when the time runs out.

1

Start a new project and delete the cat sprite. Use the paintbrush symbol to create a new blank sprite: -



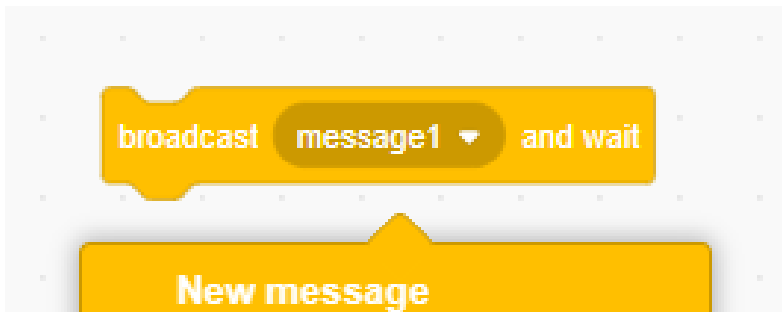
and rename it “GameLoop”.

Then make a variable called “Countdown” for the game timer and show it on stage (this happens by default – the ticked box beside the variable name is what controls this).

Build the following code to make the controlling game loop for this game. You’ll need to create new broadcast messages “Setup”, “Calculate”, “Move” and “GameOver”.

A broadcast message sends a signal to your Scratch project that other sprites and other code sections can react to in response.

You can create each message by hitting the drop down arrow and selecting “new message”:-





The idea here is the “Setup” message will signal to all sprites to get ready for the game start. It waits for them to finish, then the main loop begins. The loop sends out message tell every sprite when to run each part of its code. The loop ends only when the countdown value reaches 0, at which point the “GameOver” message is sent so all sprites can perform any final actions and the winner is announced.

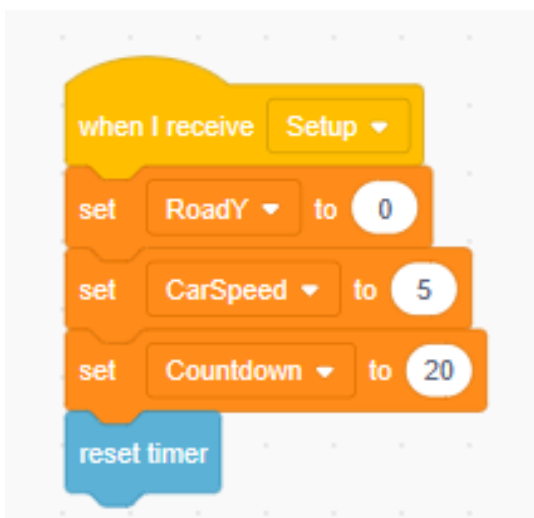
## 2

Create two new variables: -

“RoadY” to store the Y coordinate used to position our moving scenery and

“CarSpeed” to set how quickly the cars can move. Untick both variables so they don’t display on screen.

Add the following code in the GameLoop sprite as below to set starting values for these variables at the start of each game: -

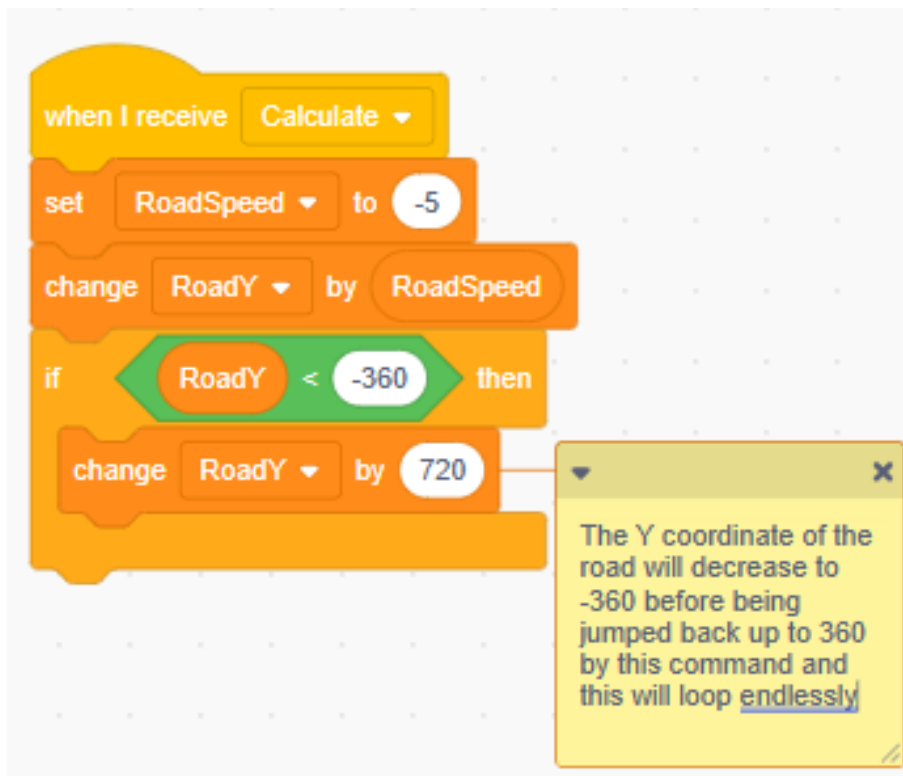


The “reset timer” block can be found in “Sensing” and it resets the internal timing counter built into Scratch.

3

Add another variable, for all sprites, called “RoadSpeed” – this will store the speed of the moving scenery. Again uncheck the tickbox so the variable is not displayed on screen.

Then add the code below to calculate the position of the road each time the game loop runs. It will be clearer how this works once you’ve made the road sprites.

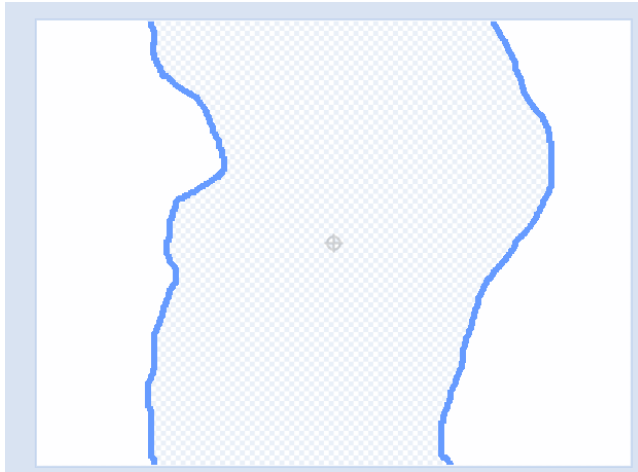


[note you do not have to add the yellow comment boxes, this is only for explanation to you]

The game will make seem as if the car sprites are moving quickly along the road, but in reality it is the road that is moving. The road will be made up of two sprites that fit seamlessly together – Road1 and Road2. These roads will take turns scrolling gown the stage, creating the illusion that the cars are constantly moving.

4

Create a new sprite and call it “Road1”. In the paint editor, click “**convert to bitmap**” option if shown then chose the paintbrush tool, set the thickness to 10 and the colour to blue. Use this to draw the edges of the road similar to the example below. When they are drawn, being careful not to leave a gap at top or bottom, use the fill tool to fill left and right with white or a colour of your choice, to create a snowy setting – it’s important to leave the middle road part transparent.

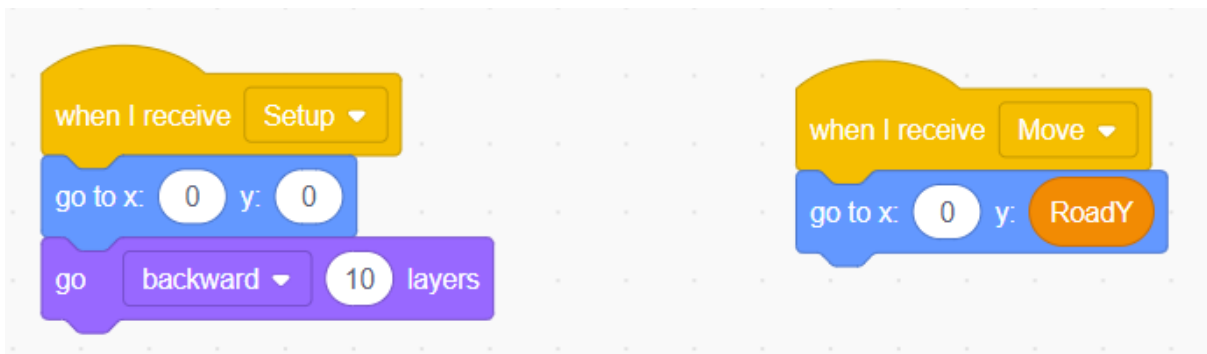


5

Now duplicate the “Road1” sprite and name the duplicate “Road2”. Go to the costumes tab of Road2 and click the “Select” tool then click on the “flip vertical” option. And the Road costume will flip upside down. The edges of Road1 and Road2 will now match as they are mirror images. They’ll look odd on screen for the moment, but we will fix this later in the coding.

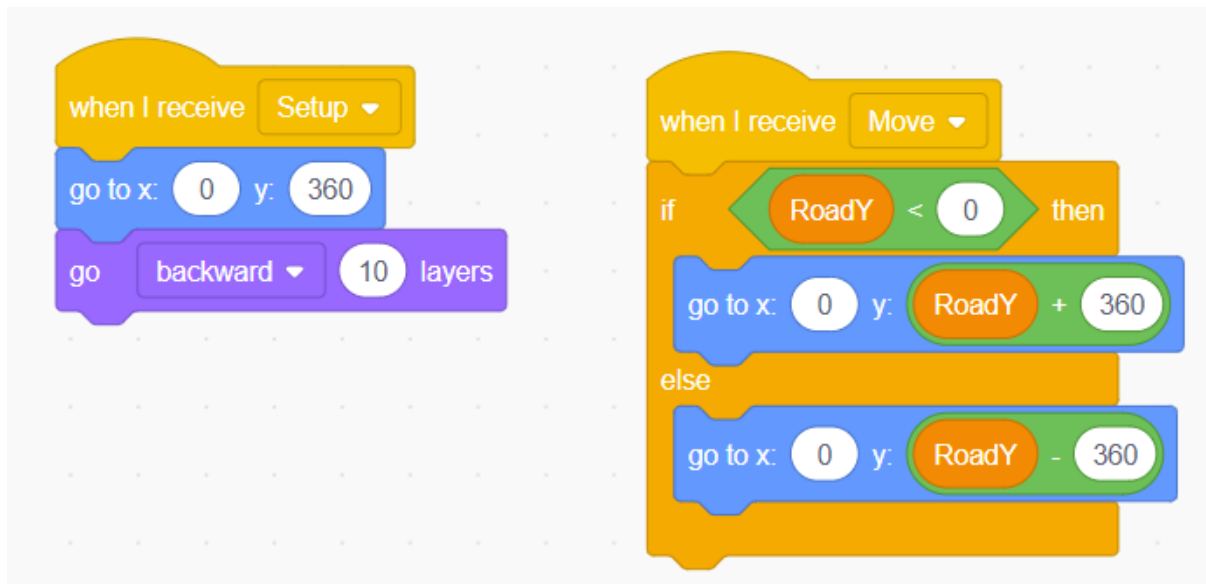
6

Add the following code blocks to Road1 to get the road moving. If you test run after adding these blocks, half the road will scroll down the screen: -



7: - Add similar code to also place and then move road2,

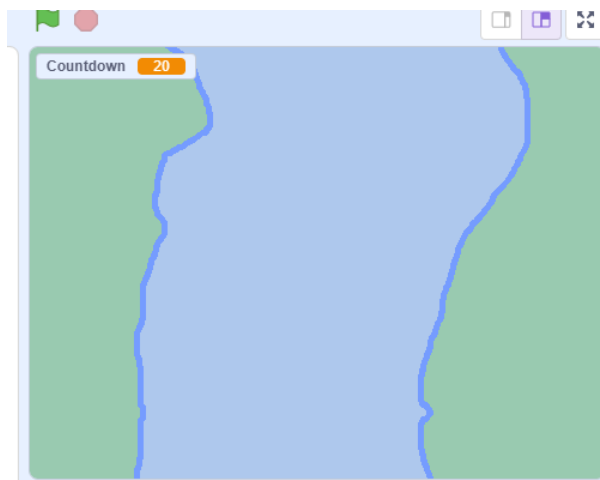
But note it very cleverly always keeps it 360 pixels away from Road1, which means they are always one full vertical screen with apart.



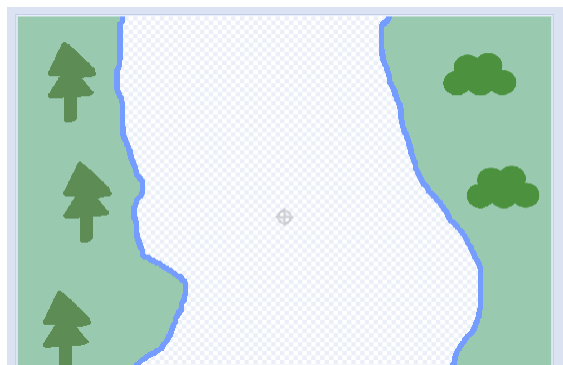
Try it and you should now have an infinitely scrolling road effect.

8

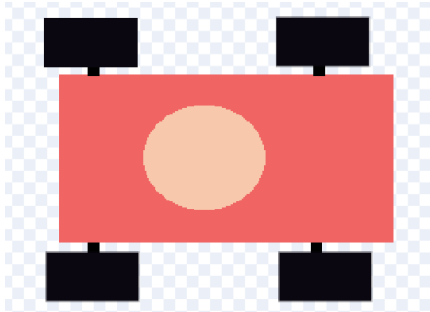
To paint the road, colour the backdrop rather than the sprite costume itself, or the cars will detect collision with the road sprites constantly. Use a light blue to suggest an icy glacier road: -



9 Add some Scenery to the side of the road, like bushes or trees: -

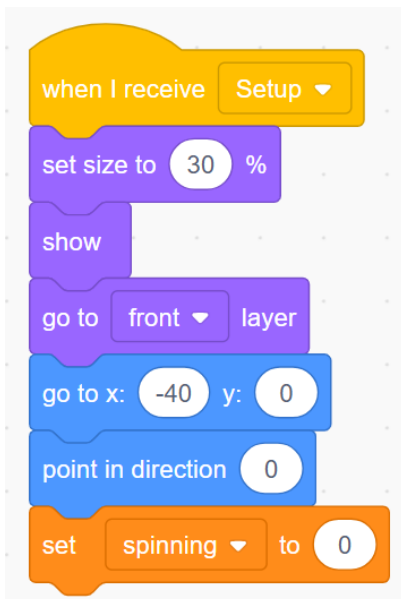


**10** Time to make a red race car sprite – start with the cat sprite from the library and use this to size the car roughly the same size. It will seem to big on stage at first but we will shrink it in the code blocks to follow. Use the paint editor and click “convert to bitmap” – delete the cat, then use the paint the tools to build something like this, centered and it must be pointing to the right as below: -



**11** Rename this sprite “RedCar”. Next create a new variable “Spinning” and make sure to select the option “for this sprite only” in the variable creation pop-up box. This means this is a variable specific to Red Car and other sprites cannot change it.

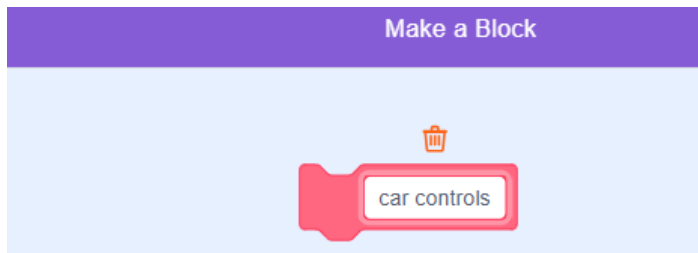
**12** Add the following code to initialise the Red Car sprite at the start of the game: -



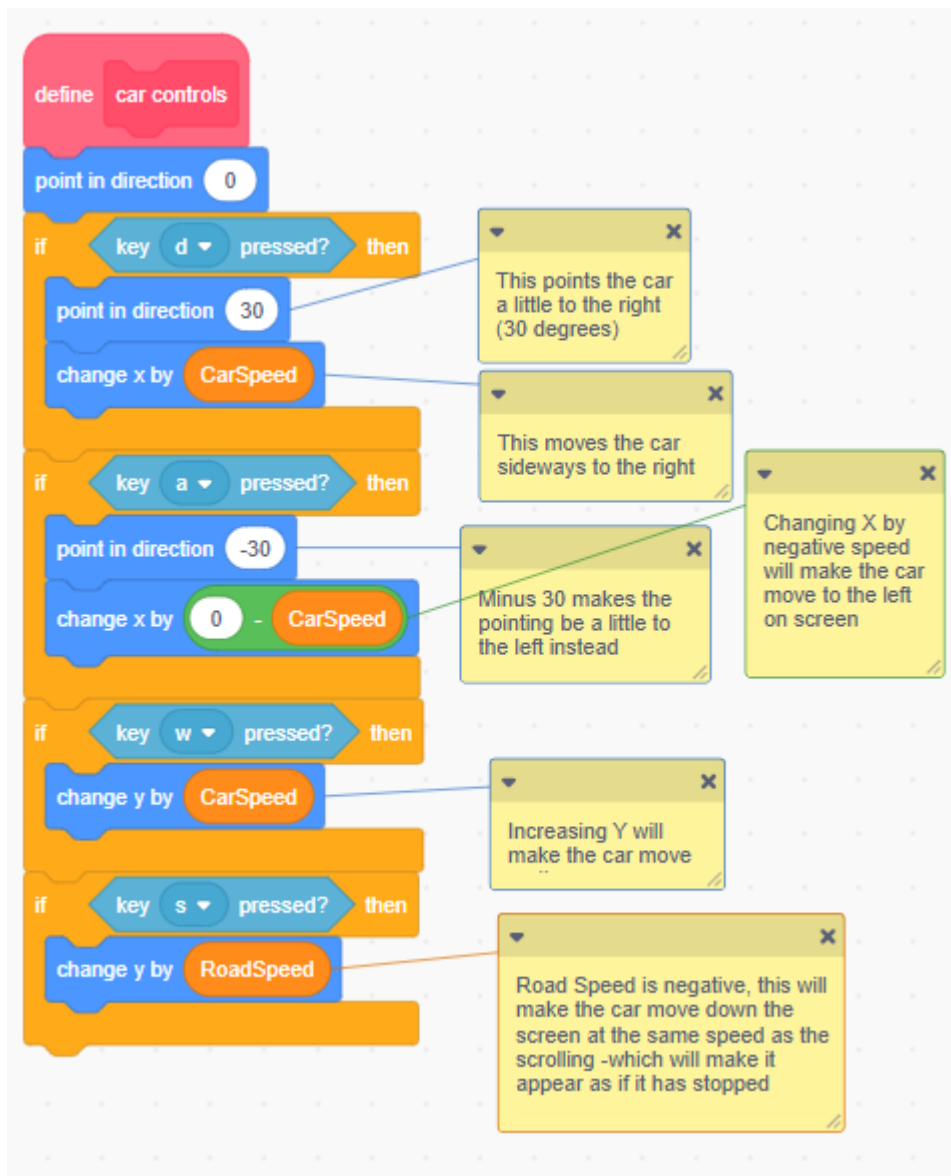
**13** Let’s define the controls to move the car:

1<sup>st</sup> create a custom block to hold the code for car controls neatly – Chose My Blocks, then make a block and call the new block “car controls”.

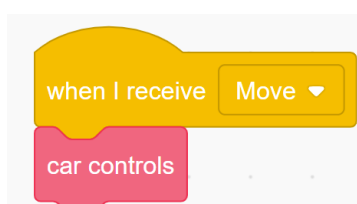
You now will have a custom define block you can use to build you custom block – pull this into your code editor and add the blocks blow to build the car control logic.



Add the following code to the define block: -



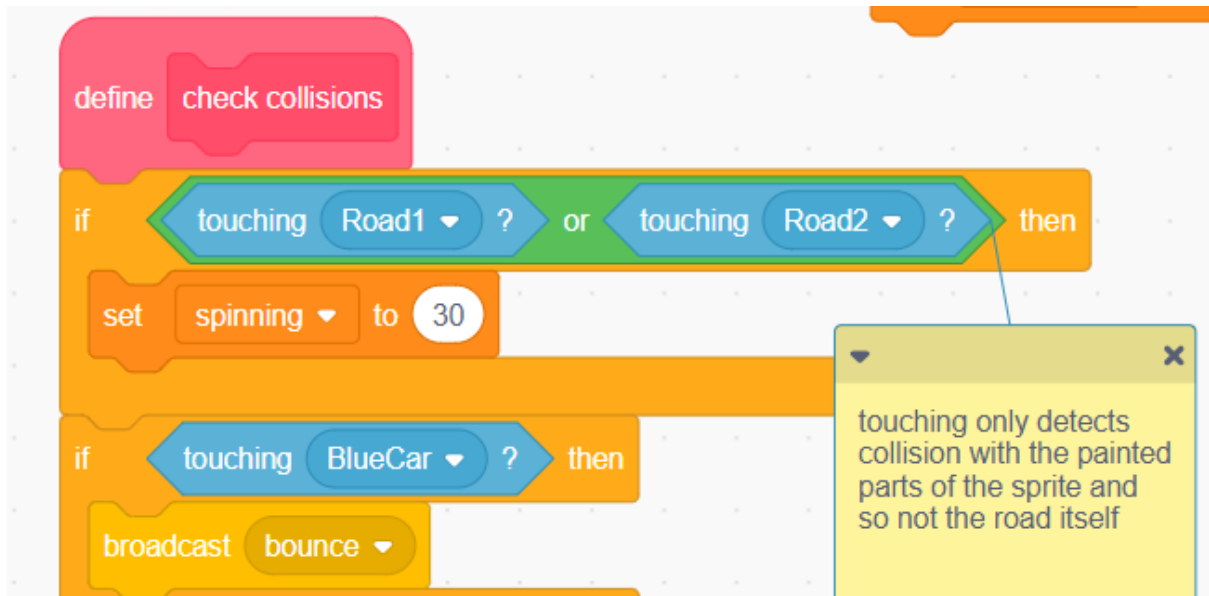
**14** This is the definition part only, for the code to be run we need to add our new custom Scratch block “car controls” into the game loop code – add it like so: -



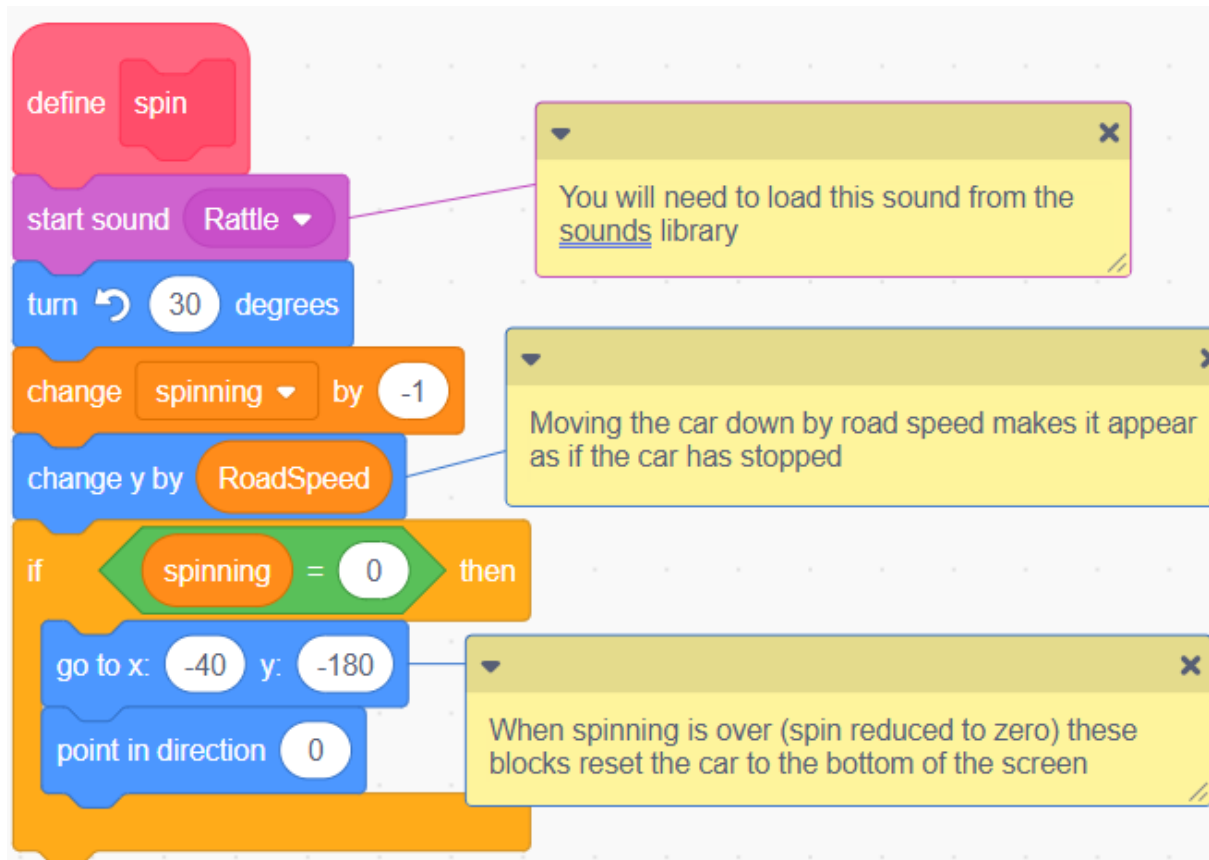


Next let's make the game more challenging, by forcing the players to avoid hitting the snow off-road.

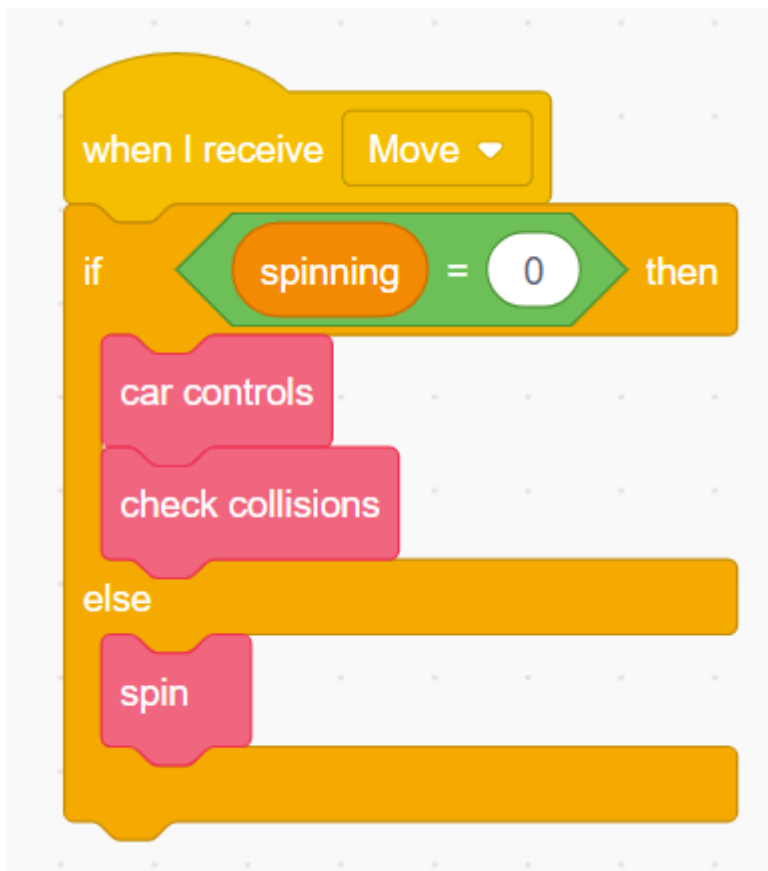
**15** Firstly, with the RedCar sprite selected, let's define another custom block as below, that will handle detecting if the car has collided with the roadside: -



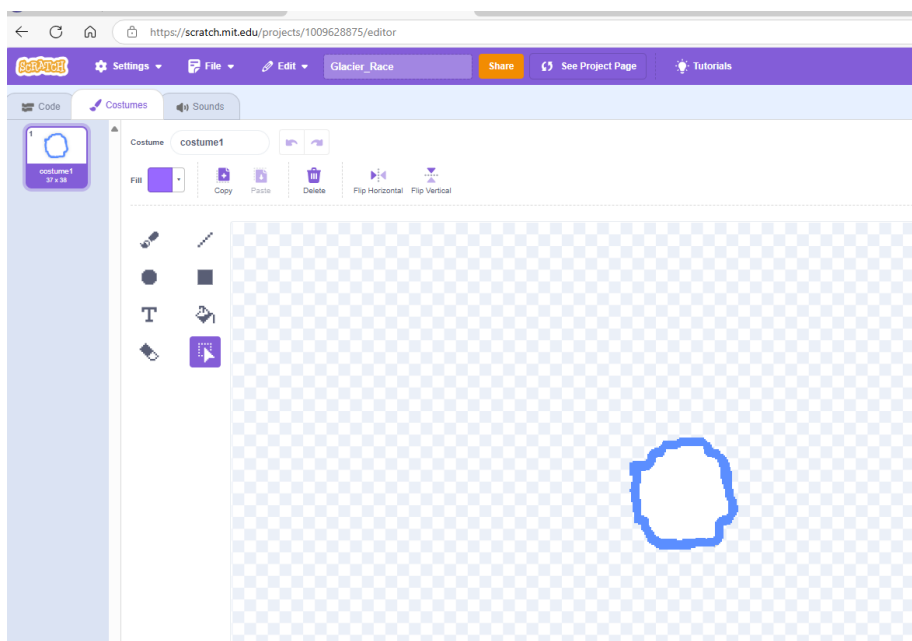
**16** Now let's define another custom block "spin" to handle making the car spin out of control whenever a collision happens: -



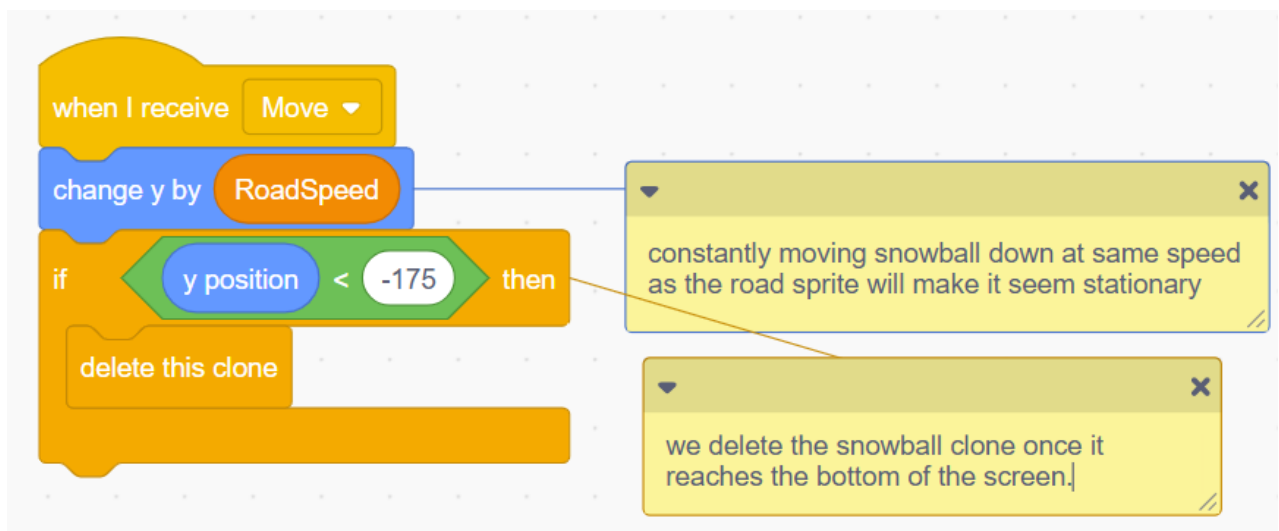
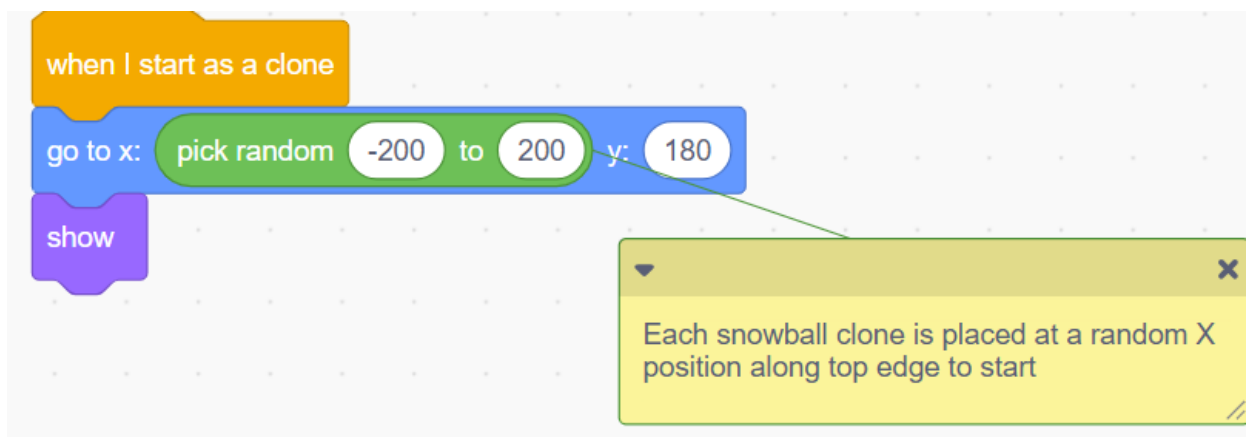
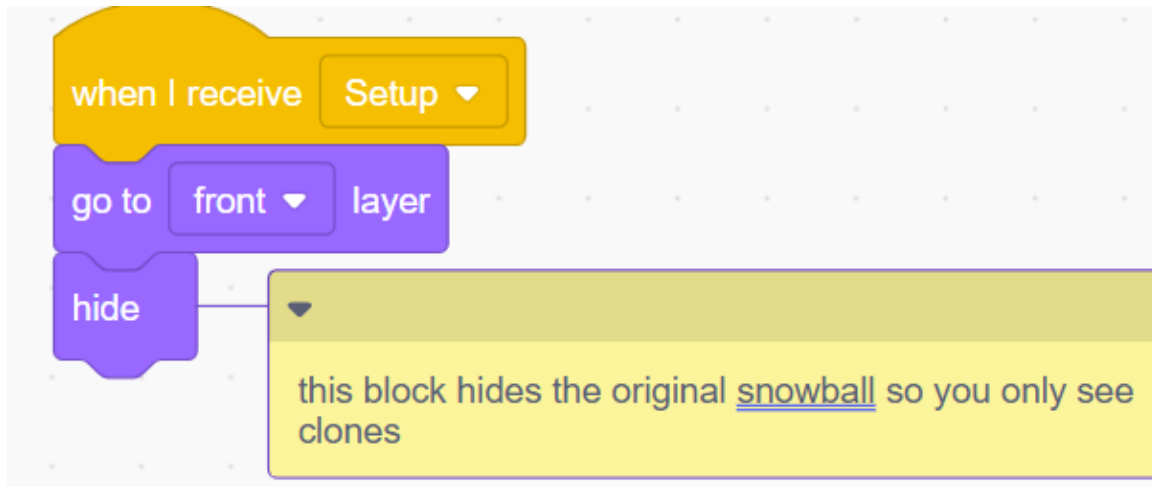
17 Update the “move” message handling as below – this will mean we only check for car controls and collisions when the car is NOT in a spin, also the car should now go into a spin if it hits the roadside: -



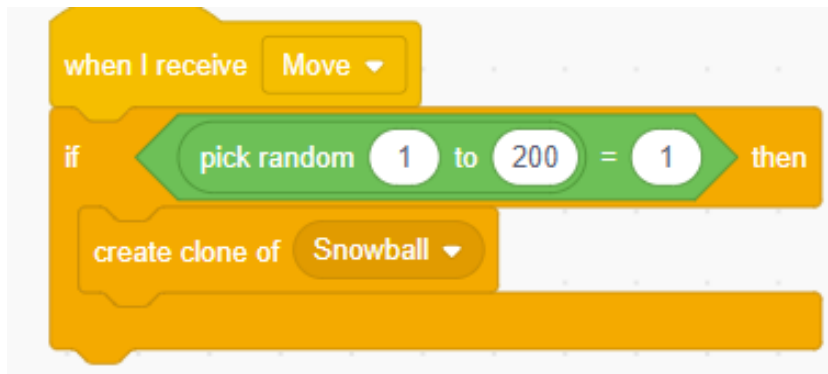
18 Lets add some snowball obstacles! Create a new sprite and name it “Snowball”. Make it about the same size as the car on stage. You can also see the size in pixels in the costume bar in the left. Fill the snowball in with white using the “Fill” tool: -



19 Add the following 3 blocks to control how the snowball will behave every time one is cloned :-



20 Now select the Game Loop sprite and add the following code to make snowballs appear at random intervals – feel free to experiment with increasing or decreasing the range to make more or less snowballs appear !



**21** To make the car collide with snowballs and go into a spin – just add it to the touching conditions check as below: -



## PLAYER TWO -!

Now we turn to coding for player two – easy – simply duplicate the red car sprite, colour its costume blue and tweak the code blocks as below: -

**22** Duplicate the red car sprite and rename it BlueCar. Note this copies all the code blocks and BlueCar even gets its own copy of the “spinning” variable [as we defined it as “for this sprite only”].

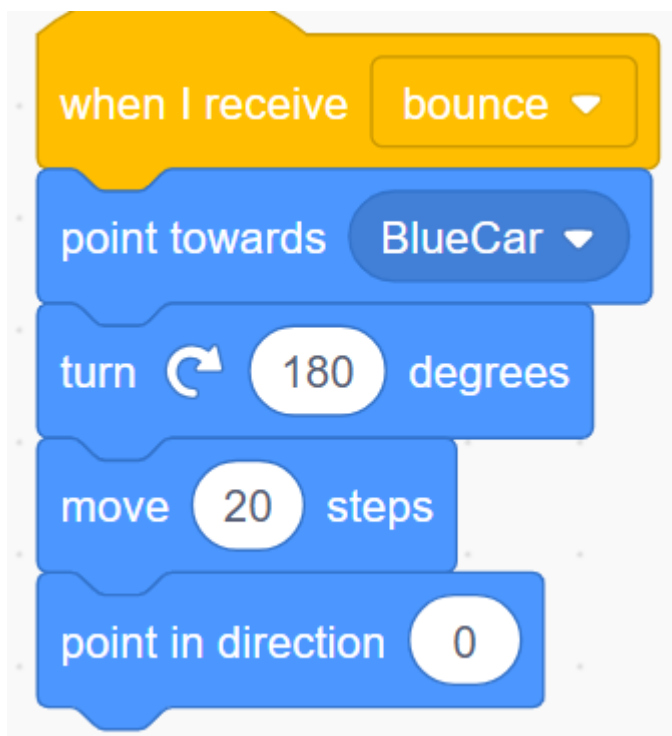
**23** Select the BlueCar sprite and click on the costumes tab and use the fill tool to change the car colour to blue.

**24** Change the X position in both the “when I receive setup” and “define spin” code below, this so the blue car is always placed just to the side of the red car when starting or restarting:



**25** In the “Define car controls” code, change the keys that control the movement of the car to the arrow keys instead. If you test the game now, both cars can be controlled but for the moment they can drive through each other!

**26** OK so to make this more realistic, let’s add code so that the cars can “sense” each other and when they collide they bounce apart. Return to the RedCar sprite and add the broadcast of a new bounce message if touching the Blue Car and then add an event to handle bouncing when the message is raised: -



Pointing towards Bluecar but then turning a half circle (i.e. 180 degrees) has the effect of pointing directly away from the blue car..

**27** Add the same code to BlueCar sprite, but change all the sprite references in the code to RedCar instead of BlueCar.

## COLLECTING GEMS

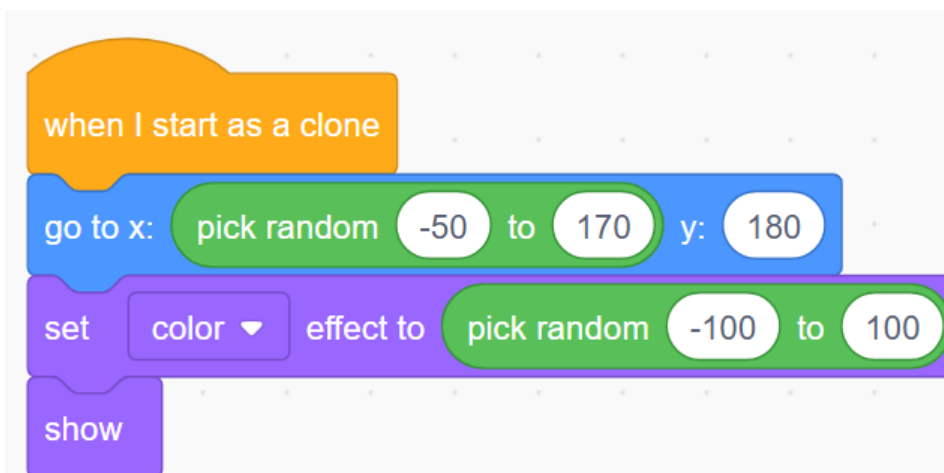
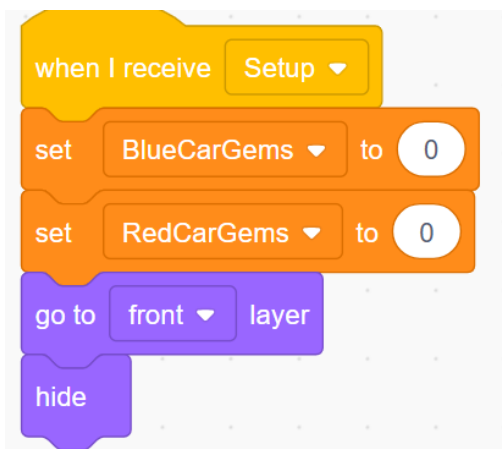
Next part of the game to add, is to add colorful gems for the 2 players to battle it out to collect. The winner will be whoever collects the most before time runs out. Again we will use the clone technique to create multiple gems at once.

**28** Create a new sprite and call it simply “Gem”. In the custom editor design something like below, 6 triangles arranged in a hexagon shape, colour each a slightly darker shade of green to get the shadow effect: -



29 Create two new variables, RedCarGems and BlueCarGems. Both can be created “for all sprites”. These will be used to keep track of how many gems each car has collected.

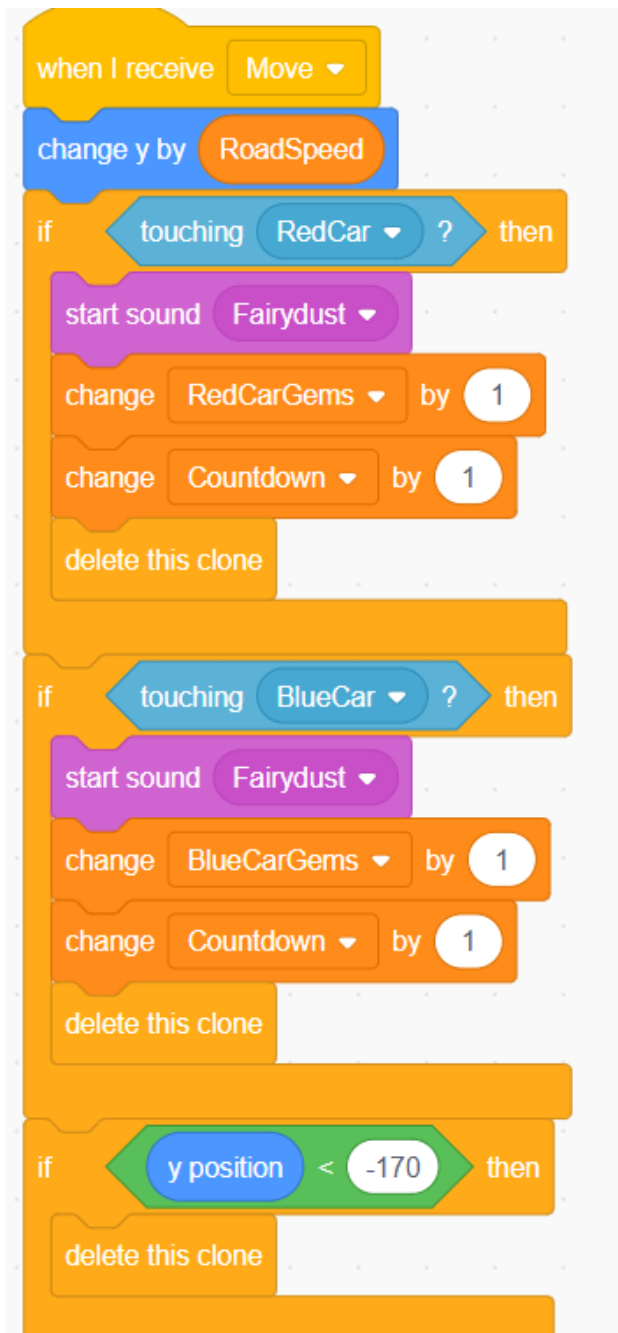
Now add the following code blocks, which are very similar to the code used to create the snowballs: -



You can experiment with the width of X value in the random range in the go to – this sets essentially how far left or how far right your gems can appear and what's best will depend on how you have designed your road sprite.

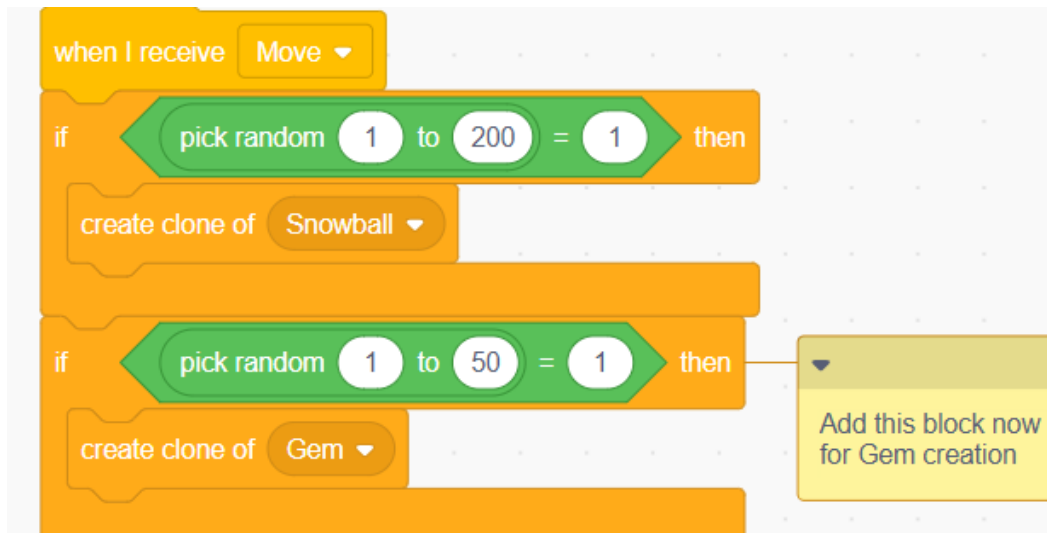
**30** Add the following code to move the Gems at roadspeed (so like the snowballs they appear stationary), and so the players can collect them and score for each. You will have to load the “fairydust” sound in the sounds tab for the Gem sprite.

Each time a gem is collected we also add 1 second to the game over countdown.

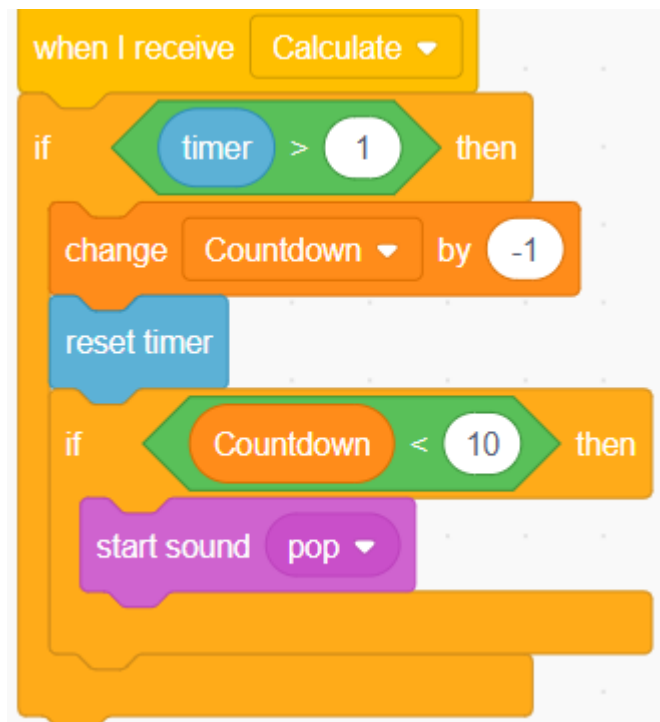


The last IF detects if the Gem has reached the bottom of the screen, and deletes the clone at this point.

**31** In the Game Loop sprite add the following code to randomly spawn Gems. You should now be able to test the game, and score by collecting Gems. The snowballs create balance in the game to discourage players from racing to the top for Gems. You can experiment with the numbers in the random range to tweak how frequently gems and snowballs appear.



**32** You may notice that the countdown isn't working and the game never ends, add the following code, to the Game Loop sprite, which uses the special "timer" variable to detect each time 1 second has passed and subtract 1 from the countdown. A "Pop" sound (will need to be loaded from the sound library) sounds for each of the last 10 seconds to warn players. And when countdown reaches zero the gameloop will exit and the game should stop:-





## PENGUIN IN CHARGE

Let's add a penguin race official to ask the players their names, start the race and announce the winner at the end.

**33** First create 4 new variables, each defined “for all sprites”. We need “RedName” and “BlueName” to store each driver’s name. And “RedInfo” and “BlueInfo” to display each driver’s score during the race. Also add a new sprite, the “Penguin 2” sprite from the library, and finally load the “Gong” sound for this new sprite.

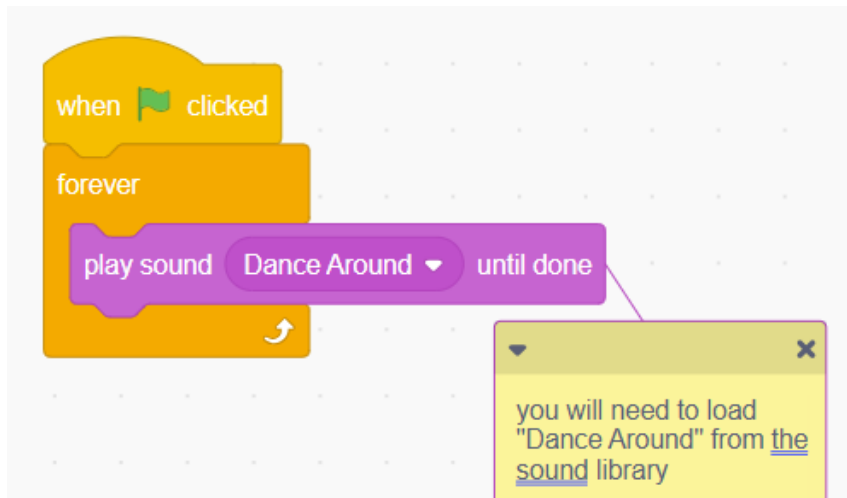
**34** Add the following code to the Penguin sprite. This will ask the players their names and show their names and current scores on screen. And because the game loop uses a “broadcast and wait” command – the race won’t start until this code has completed: -

```

when I receive Setup
  hide variable BlueInfo
  hide variable RedInfo
  go to x: -180 y: -30
  go to front layer
  show
  ask Red Driver, your controls are WASD - What is your name? and wait
  set RedName to answer
  ask Blue Driver, your controls are the arrow keys - What is your name? and wait
  set BlueName to answer
  say Go! for 1.5 seconds
  hide
  show variable BlueInfo
  show variable RedInfo
  reset timer
  
```

Whenever an "Ask" block is used, the special variable "answer" is loaded with whatever the user types in response

**38** Finally, add some rhythmic dance looping background music, add this to the **Game Loop** sprite: -

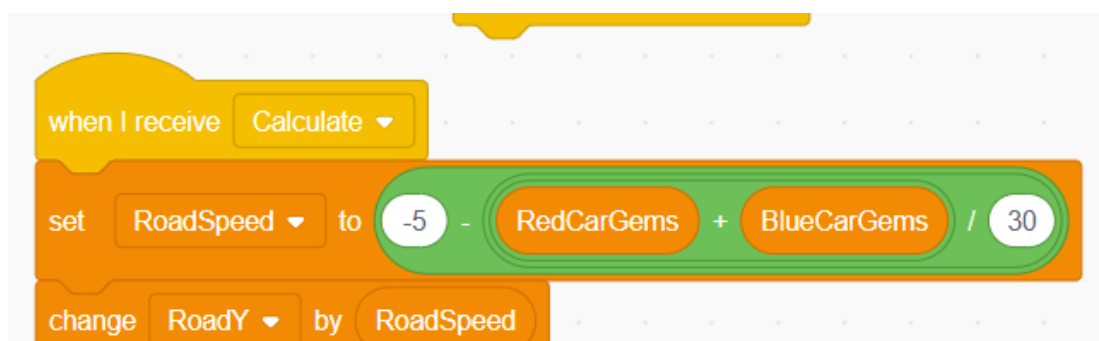


## HACKS AND TWEAKS

The fun doesn't have to end there, now it's over to you!

Some ideas for further extensions you could look at for this game:

- 1] Record your own sounds!
- 2] Write instructions in the project page for how to play the game – and publish in scratch online, if you would like to share with the world!
- 3 Fine-tuning: - Experiment with changing CarSpeed, RoadSpeed and Countdown variable settings in the code in order to finetune the game difficulty.
- 4] Change the Scenery – Redesign your setting – desert with cactuses anyone?
- 5] 1 player game – make the blue car an npc instead that just continually points and moves towards red car – do this in the car controls custom block.
- 6] For extra thrills, you can make the game speed up as each gem is collected! Something like this should do the trick:



- 7] Dirty tricks – how about collectible oil slicks or bananas you can use against your opponent?